TECHCATALYST DATA ENGINEERING 2026

SQL Foundations and Relational Databases

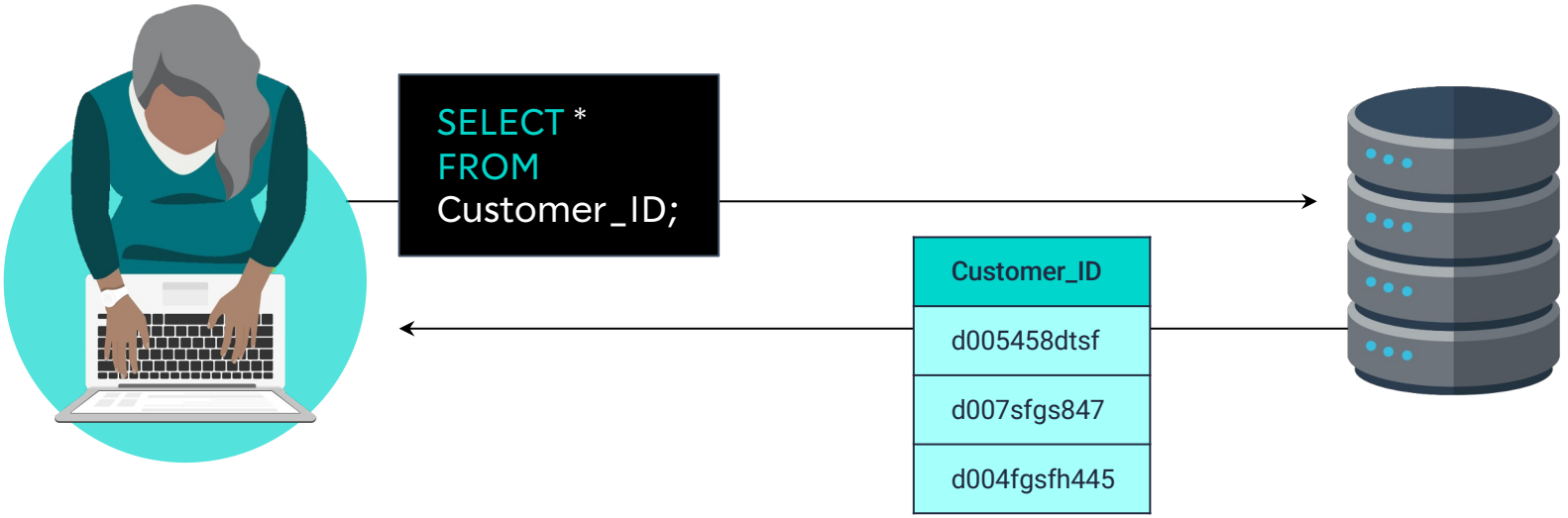
Week 4 Day 1 - the language under every warehouse, and the skill that lets you audit the machines

AI-Free Zone, Week 4 of 4. Today you earn the right to supervise the machines.

What is SQL

SQL (often pronounced "sequel") stands for Structured Query Language.

It is a powerful tool that enables programmers to create, populate, manipulate, and access databases. It also provides an easy method for dealing with server-side storage.



What is SQL

Data using SQL is stored in tables on the server, much like spreadsheets you would create in Microsoft Excel.

This makes the data easy to visualize and search.

Customer_ID	Date_ID
d005458dtsf	6/26/2019
d007sfgs847	8/3/2018
d004fgsfh445	12/3/2018

Order_ID	Customer_ID	Date_ID
10001	d005458dtsf	6/26/2019
10002	d007sfgs847	8/3/2018
10003	d004fgsfh445	12/3/2018

WHY NOW

Why learn SQL by hand in 2026

* SQL remains one of the highest-ROI skills in data. Fifty years old, still growing.
* LLMs draft SQL easily, and frequently produce plausible queries that return wrong numbers.
* Syntactically correct is not logically correct. The query runs; the answer lies.
* Coding is shifting from writing everything to editing and verifying machine output.
* You cannot audit what you cannot read. This week builds the reading skill.

> Remember: AI can draft SQL fast. Only someone who knows SQL can tell whether the answer is right.

THE VERIFICATION LAYER

Pure AI generation vs human plus SQL

AI alone

* Fast for simple asks
* Logic is blindly trusted
* Complex schemas invite bad joins and hallucinated columns
* Skill locked to one tool's interface

Human + SQL

* Slower at first, then fluent
* Logic is audited and verified
* Models the real business data accurately
* Concepts transfer to any platform

> Remember: Your job title in 2026 includes editor: verify, tweak, and optimize what the machine drafts.

EVERYWHERE

One language, every platform

Platforms in this course that speak SQL natively:

- **SQLite**
- **DuckDB**
- **BigQuery**
- **Snowflake**
- **Databricks SQL**
- **Spark SQL**
- **dbt**

$150B -> $406B

projected global DBMS market, 2026 to 2034

SQL long ago stopped being an engineering-only skill.

Marketing, product, and finance teams pull their own reports instead of waiting in the data team's queue.

FOUNDATIONS

What makes a database relational

* Data lives in tables: rows are records, columns are typed attributes.
* Tables live in schemas; schemas live in databases. Folders for tables.
* Business data is inherently relational, so the structure mirrors the business.

The shape of almost every business:

> Remember: Relationships are data too. Storing them safely is the whole point of the relational model.

BRIDGE

A table is a strict DataFrame

pandas DataFrame

* Lives in one machine's memory
* Types are suggestions; anything goes in a column
* No rules enforced; bad data loads silently
* Gone when the session ends

Relational table

* Lives in the database, any size
* Columns have declared, enforced types
* Schemas, keys, and constraints block bad data
* Persists, shared by many users at once

> Remember: pandas manipulates data. A database also protects it.

KEYS

Keys give rows an identity

* A primary key uniquely identifies each row: claim_id, customer_id, trip_id.
* No duplicates, no nulls. The database enforces this, not your discipline.
* A foreign key is another table's primary key stored as a reference.
* customer_id inside the claims table is how a claim points at its customer.
* Keys are the hooks that joins grab tomorrow. Today, just spot them in each table.

> Remember: Tomorrow: keys become joins, and the Sequel City case gets closed.

TRUST

ACID: why databases run the world's money

Atomicity

All or nothing. A transfer never debits without crediting.

Consistency

Rules always hold. No negative inventory, no orphan claim.

Isolation

Concurrent users cannot corrupt the same row.

Durability

Committed data survives the crash and the power cut.

> Remember: Banking, healthcare, and insurance run on relational databases because close enough is not acceptable.

MINDSET

Set-based thinking, not loops

Python instinct

* Loop over rows
* Check each record with an if
* Accumulate results one by one
* You describe how, step by step

SQL instinct

* Declare the result set you want
* Filters apply to all rows at once
* The engine picks the algorithm
* You describe what, the engine does how

> Remember: Stop asking how to iterate. Start describing the rows you want to keep.

ANATOMY

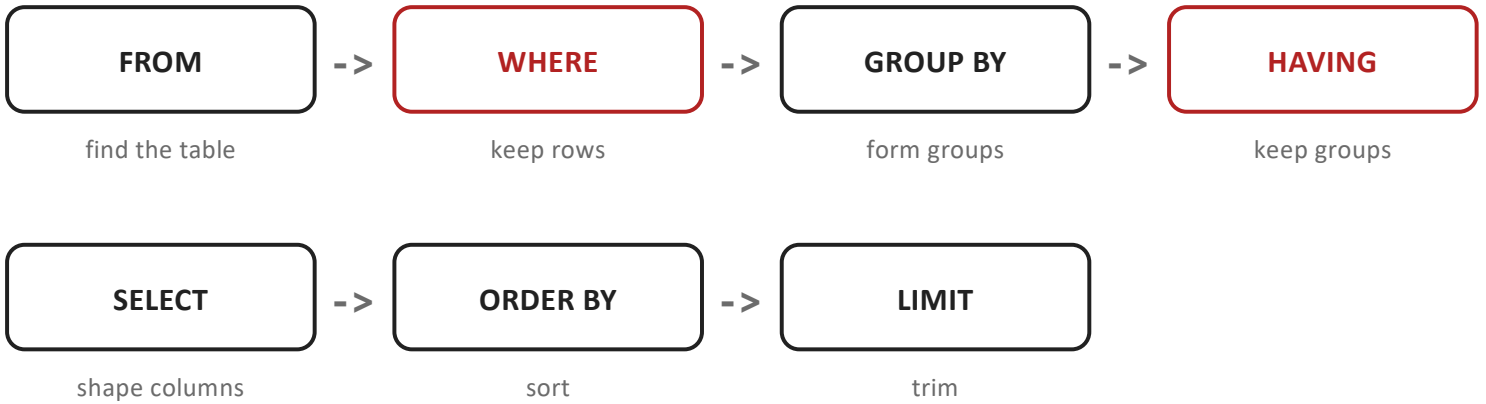
Every query you write this week

```
SELECT state, COUNT(*) AS claim_count   -- which columns, which calculations
FROM claims                             -- which table
WHERE claim_type = 'auto'               -- which rows to keep
GROUP BY state                          -- how to group the kept rows
HAVING COUNT(*) >= 2                    -- which groups to keep
ORDER BY claim_count DESC               -- how to sort the result
LIMIT 10;                               -- how many rows to show
```

> Remember: Seven clauses, fixed order, one habit: name the question in a comment above every query.

MENTAL MODEL

Writing order is not thinking order



* WHERE cannot see aggregates: it runs before groups exist. Filtering groups is HAVING's job.
* Aliases born in SELECT are not reliably visible in WHERE: SELECT happens late.

BRIDGE

The pandas dictionary

You already write this		Today it is called
`df[df["state"] == "CT"]`	->	`WHERE state = 'CT'`
`df[["claim_id", "amount"]]`	->	`SELECT claim_id, amount`
`df["state"].unique()`	->	`SELECT DISTINCT state`
`df.sort_values("a", ascending=False).head(3)`	->	`ORDER BY a DESC LIMIT 3`
`df.groupby("state").size()`	->	`GROUP BY state + COUNT(*)`
`df.groupby("t")["amt"].mean()`	->	`GROUP BY t + AVG(amt)`
`df[df["amount"].isna()]`	->	`WHERE amount IS NULL`

THE CLASSIC CONFUSION

WHERE filters rows, HAVING filters groups

WHERE

* Runs before grouping
* Sees raw column values
* WHERE state = 'CT'
* Cheaper: fewer rows reach the grouping step

HAVING

* Runs after grouping
* Sees aggregate results
* HAVING COUNT(*) > 900
* The only place a group-level condition can live

Remember: Condition mentions COUNT, SUM, or AVG? It lives in HAVING. Otherwise WHERE.

THE NEWCOMER

NULL is not zero, not empty, not equal to anything

* NULL means no value was recorded. It is the absence of an answer.
* amount = NULL matches nothing, ever. The spelling is amount IS NULL.
* Aggregates skip nulls: AVG and SUM quietly ignore missing values.
* COUNT(*) counts rows; COUNT(amount) counts recorded values only.
* When those two counts differ, you just learned something about your data.

> Remember: In insurance data, a NULL is a question someone forgot to answer. Find them before finance does.

Wildcard: % and _

Use wildcards to substitute zero, one, or multiple characters in a string.
The keyword LIKE indicates the use of a wildcard.

```
SELECT *
FROM actor
WHERE last_name LIKE 'Will%';
```

Wildcard: % and _

The % will substitute zero, one, or multiple characters in a query.
In this example, all of the following are matches: Will, Willa, and Willows.

```
SELECT *
FROM actor
WHERE last_name LIKE 'Will%';
```

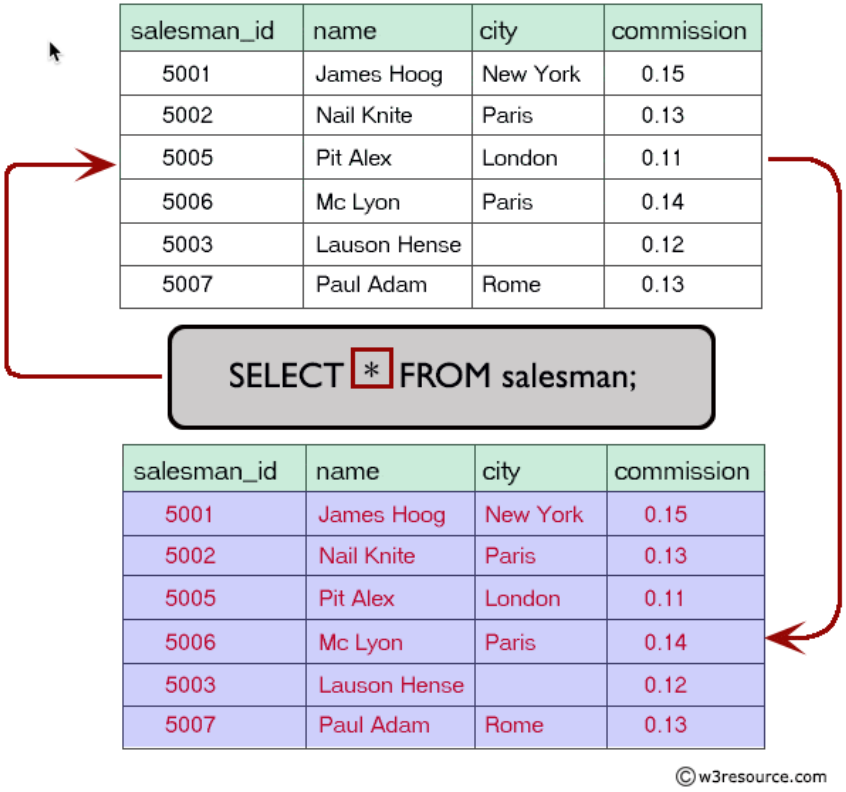
Wildcard: % and _

The _ will substitute only **one** character in a query.

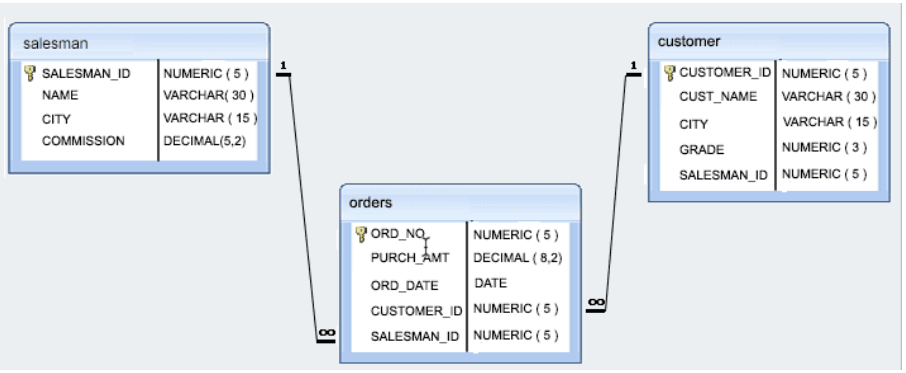
_an returns all actors whose first name contains three letters,
the second and third of which are an.

```
SELECT *
FROM actor
WHERE first_name LIKE '_an';
```

SELECT *


salesman_id name city commission
5001 James Hoog New York 0.15
5002 Nail Knite Paris 0.13
5005 Pit Alex London 0.11
5006 Mc Lyon Paris 0.14
5003 Lauson Hense 0.12
5007 Paul Adam Rome 0.13
SELECT * FROM salesman;
salesman_id name city commission
5001 James Hoog New York 0.15
5002 Nail Knite Paris 0.13
5005 Pit Alex London 0.11
5006 Mc Lyon Paris 0.14
5003 Lauson Hense 0.12
5007 Paul Adam Rome 0.13
©w3resource.com

SELECT COLUMNS IN DIFFERENT ORDER

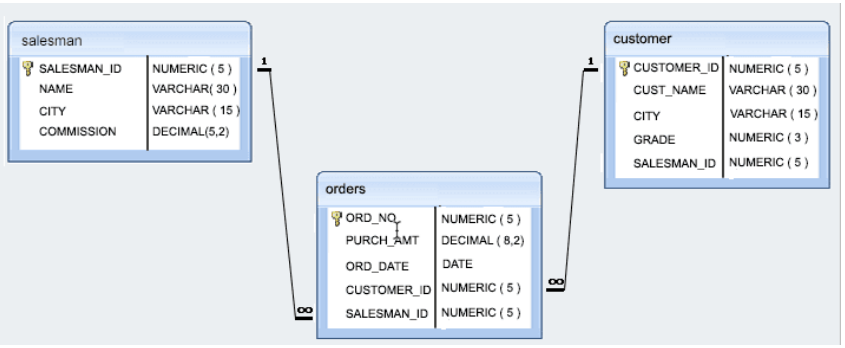


ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.50	2012-08-17	3009	5003
70007	948.50	2012-09-10	3005	5002
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.40	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.60	2012-04-25	3002	5001

```
SELECT ord_date,salesman_id,ord_no,purch_amt
FROM  orders;
```

ord_date	salesman_id	ord_no	purch_amt
2012-10-05	5002	70001	150.50
2012-09-10	5005	70009	270.65
2012-10-05	5001	70002	65.26
2012-08-17	5003	70004	110.50
2012-09-10	5002	70007	948.50
2012-07-27	5001	70005	2400.60
2012-09-10	5001	70008	5760.00
2012-10-10	5006	70010	1983.43
2012-10-10	5003	70003	2480.40
2012-06-27	5002	70012	250.45

UNIQUE VALUES

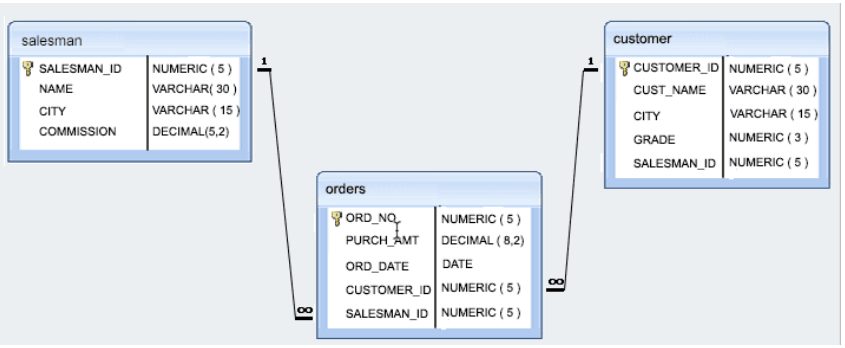


ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.50	2012-08-17	3009	5003
70007	948.50	2012-09-10	3005	5002
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.40	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.60	2012-04-25	3002	5001

```
SELECT DISTINCT salesman_id
FROM  orders;
```

salesman_id
5006
5002
5001
5005
5003
5007

SPECIFY A CONDITION



customer_id	cust_name	city	grade	salesman_id
3002	Nick Rimando	New York	100	5001
3005	Graham Zusi	California	200	5002
3004	Fabian Johnson	Paris	300	5006
3007	Brad Davis	New York	200	5001
3009	Geoff Cameron	Berlin	100	5003
3008	Julian Green	London	300	5002
3003	Jozy Altidore	Moscow	200	5007
3001	Brad Guzan	London		5005

```
SELECT * FROM customer
WHERE grade =200;
```

customer_id	cust_name	city	grade	salesman_id
3002	Nick Rimando	New York	100	5001
3005	Graham Zusi	California	200	5002
3004	Fabian Johnson	Paris	300	5006
3007	Brad Davis	New York	200	5001
3009	Geoff Cameron	Berlin	100	5003
3008	Julian Green	London	300	5002
3003	Jozy Altidore	Moscow	200	5007
3001	Brad Guzan	London		5005

Aggregate functions allow you to perform a calculation on a set of values to return a single value.

Aggregate Functions

The most commonly used aggregate functions are:

AVG	Calculates the average of a set of values
COUNT	Counts the rows in a specific table or view
MIN	Returns the minimum value in a set of values
MAX	Returns the maximum value in a set of values
SUM	Calculates the sum of a set of values

Aggregate Functions

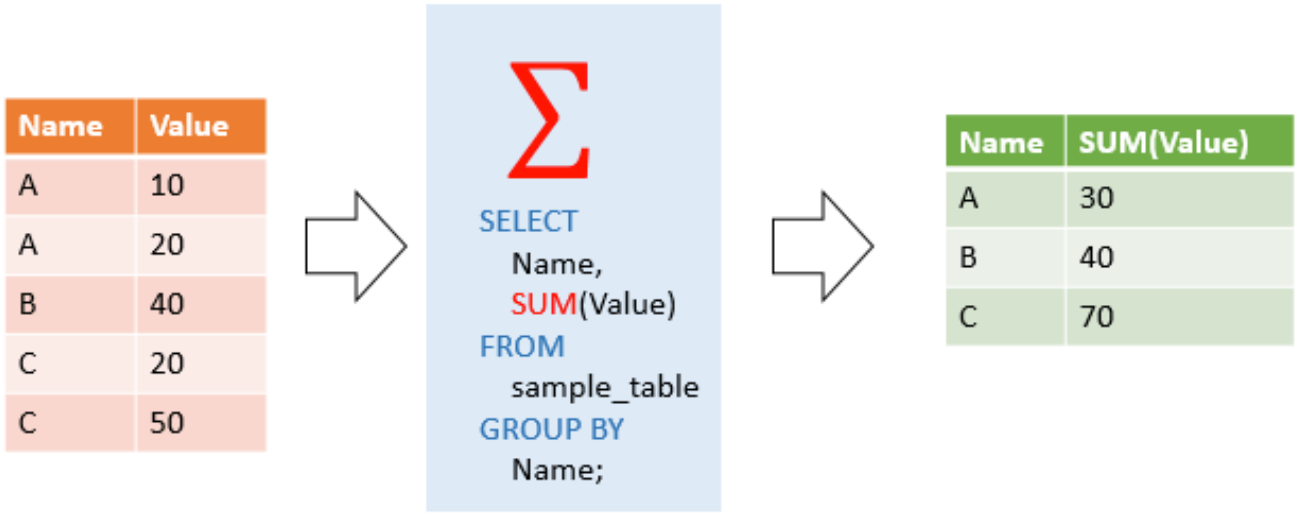
Aggregate functions are often used with:

01 The GROUP BY clause

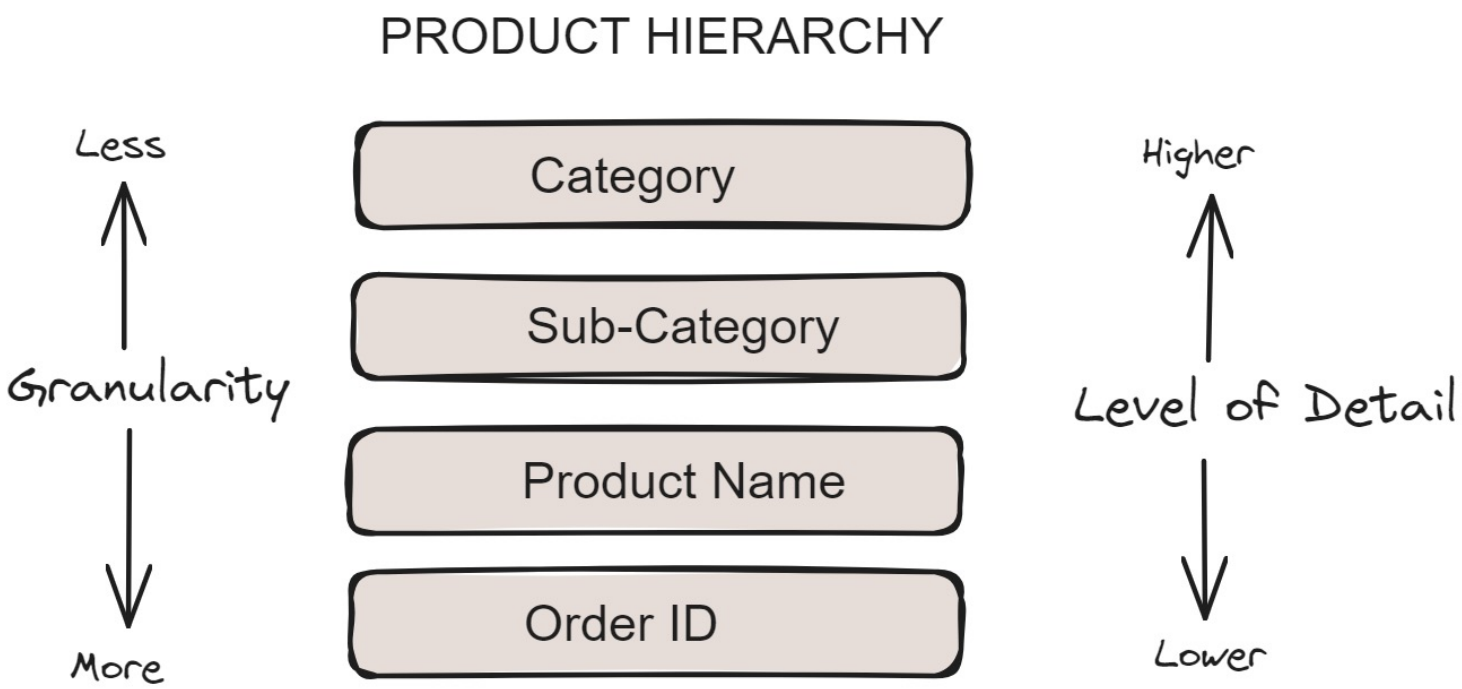
02 The HAVING clause

03 The SELECT statement

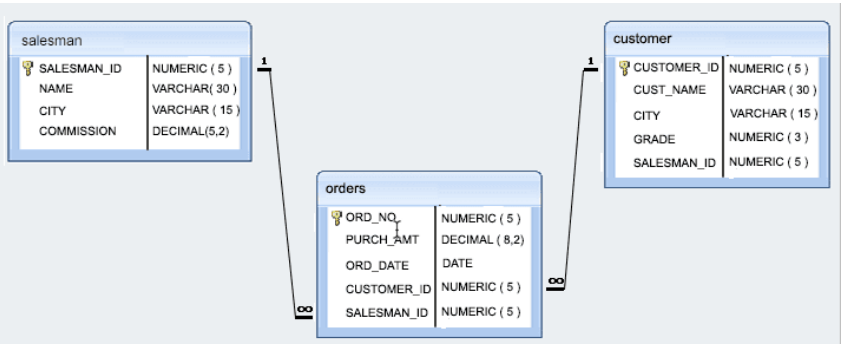
Aggregate Functions



Granularity and Aggregation



Find the total purchase amount for all orders



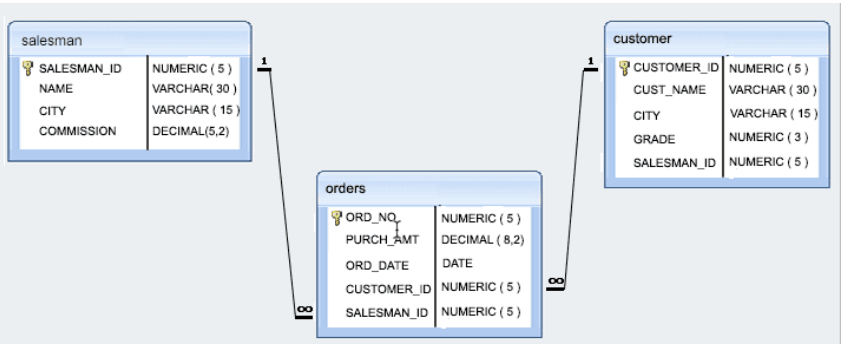
ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.50	2012-08-17	3009	5003
70007	948.50	2012-09-10	3005	5002
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.40	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.60	2012-04-25	3002	5001

```
SELECT SUM (purch_amt)
FROM orders ;
```

purch_amt
150.50
270.65
65.26
110.50
948.50
2400.60
5760.00
1983.43
2480.40
250.45
75.29
3045.60

Sum : 17541.18

Highest purchase amount ordered by the each customer



ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70009	270.65	2012-09-10	3001	5005
70002	65.26	2012-10-05	3002	5001
70004	110.50	2012-08-17	3009	5003
70007	948.50	2012-09-10	3005	5002
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70010	1983.43	2012-10-10	3004	5006
70003	2480.40	2012-10-10	3009	5003
70012	250.45	2012-06-27	3008	5002
70011	75.29	2012-08-17	3003	5007
70013	3045.60	2012-04-25	3002	5001

```
SELECT customer_id, MAX(purch_amt)
FROM orders
GROUP BY customer_id ;
```

customer_id	purch_amt
3005	150.5
3001	270.65
3002	65.26
3009	110.5
3005	948.5
3007	2400.6
3002	5760
3004	1983.43
3009	2480.4
3008	250.45
3003	75.29
3002	3045.6

customer_id	max
3004	1983.43
3008	250.45
3001	270.65
3007	2400.6
3005	948.5
3002	5760
3009	2480.4

Order By Aggregates

The ORDER BY function:

- Is added towards the end of a query.
- Returns in an ascending order by default.
- Can also return in a descending order by adding DESC.
- Can limit the return by adding LIMIT.
- **NOTE:** Use the ROUND function to round up the number after the the decimal.

TODAY

The mission: three databases, one skill

Warmup: pandas -> SQLite -> **LeBron: filter and sort** -> **Call center: aggregate**

...and after the break:

Sequel City: the investigation opens

Filters, grouped counts, CASE, and a suspect list. The case does not close today.

* Check Yourself tables give expected counts: match them or explain why not.
* Partner check after each activity; the post-quiz closes the day.

> Remember: Everything is written by hand this week. That is the point.

THE WEEK

From local file to cloud warehouse in five days

Mon	Tue	Wed	Thu	Fri
SQLite	**SQLite + BigQuery**	**BigQuery + DuckDB**	**Snowflake**	**Snowflake + S3**
core SQL, local and free	keys and joins	subqueries, CTEs, OLAP	CTAS, views, table types	COPY INTO, end to end

> Remember: Same seven clauses on every platform. Learn them once today, reuse them for a career.

RECAP

What you can say after today

* SQL is the language of every warehouse, and the audit skill behind AI-drafted queries.
* A relational table is a strict, persistent, shared DataFrame with enforced rules.
* Seven clauses in a fixed thinking order answer most business questions.
* WHERE trims rows, HAVING trims groups, IS NULL finds the silent gaps.
* Tomorrow: keys become joins, and the Sequel City case gets closed.

> Remember: Post-quiz now, then the optional BigQuery homework if you want tonight's stretch.